Recorrido del código

`highOrderElectroActivationFoamImplicitPETScDistributed`

Generado el 2026-10-18 a partir de
`highOrderElectroActivationFoamImplicitPETScDistributed.C` (5555 líneas) y de los headers de `src/monodomainCommon`

Cómo leer esto. Los listados NO son copias: salen de `\lstinputlisting` apuntando al fuente real, con rangos que este reporte recalcula al regenerarse. El PDF de hoy muestra el código de hoy. Los números de línea del margen son los del archivo.

Índice

1. Qué resuelve este solver, y dónde vive el código **2**

2. El modelo iónico TNNP **2**

3. Los integradores de EDO y el driver de Vm **4**

4. Esquemas temporales: theta, BDF1-4 y el arranque ESDIRK3 **6**

5. Reconstrucciones en los puntos de Gauss **9**

6. El residuo de estados: una colectiva que colgaba **11**

7. El criterio de parada no lineal, común a todos los métodos **12**

8. Bloques de construcción del ensamblado **13**

9. Las dos matrices de rigidez, y el defecto de la conductividad **14**

10. La solución lineal: PETSc con precondicionado por derecha **15**

11. Modo híbrido: GP en el frente, CC en el resto **16**

12. Tiempos de activación y muestreo del benchmark **18**

13. main(): montaje y lazo temporal **20**

14. Índice de funciones **20**

1. Qué resuelve este solver, y dónde vive el código

Resuelve la propagación del potencial de acción cardíaco: la ecuación de monodominio acoplada al modelo iónico **ten Tusscher–Noble–Noble–Panfilov (TNNP)**, con el objetivo de reproducir el benchmark de Niederer et al. (2012). A diferencia del solver MMS, acá **no hay solución exacta**: la verificación es por convergencia de Cauchy, por invariancia frente a la descomposición y contra referencia publicada.

En cada paso de tiempo hay dos trabajos muy distintos:

1. integrar, en cada celda o punto de Gauss, un sistema **rígido** de 19 EDO — el modelo TNNP — para obtener la corriente iónica I_{ion};
2. resolver el sistema lineal implícito de la difusión.

El primero domina el tiempo. **MPI** reparte la malla entre ranks y toca todas las fases; **OpenMP** reparte, dentro de un rank, el lazo de EDO por punto y las reconstrucciones en los puntos de Gauss (de V_m y de los estados), además de los productos matriz–vector de Eigen.

Dónde vive el código (2026-10). Todo lo que el solver comparte con el MMS — álgebra distribuida y PETSc, ensamblado de masa y rigidez, reconstrucciones, GMRES/JFNK, esquemas temporales, criterio de parada — está en `src/monodomainCommon/`, en headers que se incluyen *dentro* del espacio de nombres anónimo del `.C`: no hay biblioteca que enlazar, y un cambio ahí cambia los dos solvers. En el `.C` queda lo propio: el modelo TNNP, el estímulo, el modo híbrido, los tiempos de activación y el lazo temporal. Cada listado de este recorrido indica de qué archivo sale.

2. El modelo iónico TNNP

El modelo vive en una clase dentro del archivo, con sus 19 estados y sus variables algebraicas. `computeRates` evalúa las derivadas; `computeVariables` las cantidades algebraicas; `calculateCurrent` devuelve I_{ion}.

`protectState` es la protección numérica: acota compuertas a $[0, 1]$, pone pisos a las concentraciones y a V_m. Lleva un contador de correcciones, el único estado compartido de todo el camino de las EDO — por eso su incremento va bajo `omp critical`. **Está apagada por defecto**, en el solver y desde 2026-10-16 también en el harness: encendida, su contador llega al tope de 10^6 y los BDF pierden el orden (E02: con estados CC, BDF2/3/4 dan 1.74/2.22/3.54 con ella y 1.89/2.96/4.60 sin ella), y ninguna corrida la necesitó.

El test **E26** verifica el modelo aislado, en 0-D, contra los valores publicados de ten Tusscher 2004 (el pico, contra el propio modelo convergido).

```
849         void protectState
850         (
851             scalarField& state,
852             const scalar,
853             const label,
854             const char*
855         )
856         {
857             if (!numericalProtection_) return;
858 
859             auto correct = [&](const label i, const scalar value)
860             {
861                 if (state[i] != value)
862                 {
863                     state[i] = value;
864                     recordCorrection();
865                 }
866             };
```

```
            if (!std::isfinite(state[V]))
            {
                correct(V, -86.2);
            }
            if (clampVmForRates_)
            {
                correct(V, min(max(state[V], vmMinForRates_), vmMaxForRates_));
            }
            if (mag(state[V]) < vmZeroEps_)
            {
                correct(V, state[V] < 0.0 ? -vmZeroEps_ : vmZeroEps_);
            }

            const label concentrationIDs[] = {K_i, Na_i, Ca_i, Ca_SR};
            for (const label id : concentrationIDs)
            {
                if (!std::isfinite(state[id]) || state[id] < concentrationFloor_)
                {
                    correct(id, concentrationFloor_);
                }
            }

            if (clampGates_)
            {
                const label gateIDs[] = {Xr1, Xr2, Xs, m, h, j, d, f, fCa, s, r, g};
                for (const label id : gateIDs)
                {
                    if (!std::isfinite(state[id]))
                    {
                        correct(id, 0.0);
                    }
                    else
                    {
                        correct(id, min(max(state[id], scalar(0.0)), scalar(1.0)));
                    }
                }
            }
        }
```

Listing 1: La protección numérica y su contador compartido. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

```
        void calculateCurrent
        (
            const scalar stepStartTime,
            const scalar,
            const scalarField& Vm,
            scalarField& Im,
            Field<Field<scalar>>& states
        )
        {
            const scalar tStartMs = 1000.0*stepStartTime;
            if (Im.size() != Vm.size())
            {
                FatalErrorInFunction
                    << "Im.size() != Vm.size()" << abort(FatalError);
            }

            // A mismatch here is a programming error, not something to repair
            // on the fly: re-initialising would discard the accumulated state
            // history. The state array must already be sized to the point set
            // it is integrated on (mesh.nCells() for cellCentredReconstruct,
            // the Ion Gauss points otherwise).
            if (states.size() != Vm.size())
            {
                FatalErrorInFunction
                    << "states.size() = " << states.size()
                    << " != Vm.size() = " << Vm.size() << nl
                    << "The ionic state array must be sized to the point set "
                    << "it is integrated on." << abort(FatalError);
            }

            forAll(Vm, pointI)
            {
```

```
                scalarField& y = states[pointI];
                y[V] = 1000.0*Vm[pointI];
                protectState(y, tStartMs, pointI, "calculateCurrent");
                computeVariables(tStartMs, y, rates_[pointI], algebraic_[pointI]);
                Im[pointI] = algebraic_[pointI][Iion_cm];
            }
        }
```

Listing 2: La corriente iónica. Recortada. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]`

3. Los integradores de EDO y el driver de Vm

El solver trae sus propios steppers: `eulerStep`, `rk4Step` y `rkf45Trial` adaptativo. Un nombre desconocido es `FatalError`. Cada integración RKF45 arranca desde `stateODEInitialStep` (el viejo `stepMs_` que heredaba el paso entre llamadas, y hacía inalcanzable la clave, ya no existe). **E27** hace converger los tres integradores al mismo potencial de acción.

Dentro del paso, la EDO no ve un V_m congelado: `computeRates` lo toma del *driver* (`drivenVmMilliVolt`), un polinomio en el tiempo que pasa por V^n y el candidato V^{n+1} — y, con `stateODEDriver history`, también por los niveles anteriores de la historia BDF o por los valores de etapa ESDIRK. Con V_m congelado el TNNP daba orden 1 incluso con Crank–Nicolson; con la recta, 2; la historia es la que deja a BDF3 y BDF4 dar su orden.

`solveODE` reparte los puntos con OpenMP; cada punto es una EDO independiente, así que el resultado es bit a bit igual con cualquier cantidad de hilos (**E33**).

```
        void rkf45Trial
        (
            const scalar tMs,
            const scalar hMs,
            const scalarField& y,
            scalarField& yFifth,
            scalarField& rates,
            scalarField& algebraic,
            const label pointI,
            scalar& err,
            const VmDriverNodes* nodes,
            const scalar t0Ms,
            const scalar dtMs
        )
        {
            scalarField k1(NUM_STATES, 0.0), k2(NUM_STATES, 0.0), k3(NUM_STATES, 0.0);
            scalarField k4(NUM_STATES, 0.0), k5(NUM_STATES, 0.0), k6(NUM_STATES, 0.0);
            scalarField yt(NUM_STATES, 0.0), yFourth(NUM_STATES, 0.0);

            computeRates(tMs, y, k1, algebraic, pointI, "RKF45_k1", nodes, t0Ms, dtMs);
            forAll(y, i) yt[i] = y[i] + hMs*(1.0/5.0)*k1[i];
            computeRates(tMs + hMs/5.0, yt, k2, algebraic, pointI, "RKF45_k2", nodes, t0Ms, dtMs);
            forAll(y, i) yt[i] = y[i] + hMs*((3.0/40.0)*k1[i] + (9.0/40.0)*k2[i]);
            computeRates(tMs + 3.0*hMs/10.0, yt, k3, algebraic, pointI, "RKF45_k3", nodes, t0Ms, dtMs);
            forAll(y, i) yt[i] = y[i] + hMs*((3.0/10.0)*k1[i] - (9.0/10.0)*k2[i] + (6.0/5.0)*k3[i]);
            computeRates(tMs + 3.0*hMs/5.0, yt, k4, algebraic, pointI, "RKF45_k4", nodes, t0Ms, dtMs);
            forAll(y, i) yt[i] = y[i] + hMs*((-11.0/54.0)*k1[i] + (5.0/2.0)*k2[i] - (70.0/27.0)*k3[i] +
    (35.0/27.0)*k4[i]);
            computeRates(tMs + hMs, yt, k5, algebraic, pointI, "RKF45_k5", nodes, t0Ms, dtMs);
            forAll(y, i) yt[i] = y[i] + hMs*((1631.0/55296.0)*k1[i] + (175.0/512.0)*k2[i] + (575.0/13824.0)*k3[
    i] + (44275.0/110592.0)*k4[i] + (253.0/4096.0)*k5[i]);
            computeRates(tMs + 7.0*hMs/8.0, yt, k6, algebraic, pointI, "RKF45_k6", nodes, t0Ms, dtMs);

            err = 0.0;
            forAll(y, i)
            {
                yFifth[i] = y[i] + hMs*((37.0/378.0)*k1[i] + (250.0/621.0)*k3[i] + (125.0/594.0)*k4[i] +
    (512.0/1771.0)*k6[i]);
                yFourth[i] = y[i] + hMs*((2825.0/27648.0)*k1[i] + (18575.0/48384.0)*k3[i] + (13525.0/55296.0)*
    k4[i] + (277.0/14336.0)*k5[i] + 0.25*k6[i]);
                const scalar scale = odeAbsTol_ + odeRelTol_*max(mag(yFifth[i]), mag(yFourth[i]));
```

```
                err = max(err, mag(yFifth[i] - yFourth[i])/max(scale, SMALL));
            }
            rates = k6;
        }
```

Listing 3: El paso adaptativo. Recortado. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

```
        static scalar drivenVmMilliVolt
        (
            const VmDriverNodes& nodes,
            const scalar tMs,
            const scalar t0Ms,
            const scalar dtMs
        )
        {
            const scalar s = (dtMs > SMALL) ? (tMs - t0Ms)/dtMs : 1.0;
            return 1000.0*interpolateVmDriver(nodes, s);
        }
```

Listing 4: El driver de Vm dentro del paso. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

```
    inline scalar interpolateVmDriver
    (
        const VmDriverNodes& nd,
        const scalar s
    )
    {
        // Written out for two nodes: the path every one-step scheme takes.
        if (nd.nNodes <= 2)
        {
            return (1.0 - s)*nd.v[0] + s*nd.v[1];
        }

        scalar result = 0.0;

        for (label j = 0; j < nd.nNodes; ++j)
        {
            scalar Lj = 1.0;

            for (label m = 0; m < nd.nNodes; ++m)
            {
                if (m != j)
                {
                    Lj *= (s - nd.s[m])/(nd.s[j] - nd.s[m]);
                }
            }

            result += Lj*nd.v[j];
        }

        return result;
    }
```

Listing 5: La interpolación de Lagrange del driver (compartida). [monodomainTimeSchemes.H]

```
        void solveODE
        (
            const scalar stepStartTime,
            const scalar deltaT,
            const scalarField& Vm,
            scalarField& Im,
            Field<Field<scalar>>& states,
            const labelUList& scratchIDs = emptyScratchIDs(),
            const VmDriverStages* driver = nullptr
        )
        {
            const scalar tStartMs = 1000.0*stepStartTime;
            const scalar dtMs = 1000.0*deltaT;

            // Added for cardiacFoam: the Vm driver, when given, holds one field
            // per node except the last, which is Vm itself (the candidate). A
            // missing or mis-sized node is a silently wrong ODE driver rather
```

```
            // than a crash, so it is checked here, before the parallel loop.
            if (driver)
            {
                if (driver->nNodes < 2 || driver->nNodes > driver->s.size())
                {
                    FatalErrorInFunction
                        << "Vm driver with " << driver->nNodes << " nodes."
                        << abort(FatalError);
                }

                for (label j = 0; j < driver->nNodes - 1; ++j)
                {
                    if
                    (
                        !driver->field[j]
                     || driver->field[j]->size() != Vm.size()
                    )
                    {
                        FatalErrorInFunction
                            << "Vm driver node " << j << " of "
                            << driver->nNodes << " is not set or is not sized"
                            << " to the " << Vm.size() << " points integrated."
                            << abort(FatalError);
                    }
                }
            }
            if (Im.size() != Vm.size())
            {
                FatalErrorInFunction
                    << "Im.size() != Vm.size()" << abort(FatalError);
            }

            // Added for cardiacFoam: the caller may pass an explicit scratch
            // slot for each point. This is what keeps the front-aware hybrid
            // correct. That path calls solveODE twice per iteration, once with
            // a compacted array of nF front Gauss points and once with a
            // compacted array of nS smooth cells, both indexed from 0. Without
            // a slot map both passes write algebraic_/rates_[0..n),
            // so the second silently overwrites the first's scratch even
            // though the two arrays describe different physical points.
            if (!scratchIDs.empty())
            {
                if (scratchIDs.size() != Vm.size())
```

Listing 6: El despacho de stepper y la región OpenMP. Recortado. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

4. Esquemas temporales: theta, BDF1-4 y el arranque ESDIRK3

`timeSchemeCoeffs` da los coeficientes de backwardEuler, crankNicolson y BDF1–BDF4. Un BDF-k necesita $k-1$ niveles previos: los da un arranque **ESDIRK3(2)4L[2]SA** (`esdirk3Tableau`), de tercer orden respecto del sistema semidiscreto, que es lo que conserva el orden nominal.

La historia se **reinicia** (y los $k-1$ pasos siguientes vuelven a ser ESDIRK3) cuando cambia el estímulo — un salto en el forzado que un método multipaso no puede cruzar — en un paso revertido, y en un paso de longitud no nominal. El estímulo se promedia sobre el paso (`applyStimulus`), así que la carga inyectada es exacta para cualquier Δt; antes duraba un paso de más.

E02 mide el orden de los seis esquemas en cuatro caminos de código (bajo orden, CC, híbrido y GP). En el par más fino, CN da 2.00 y BDF4 entre 4.6 y 5.6.

```
    TimeSchemeCoeffs timeSchemeCoeffs(const word& scheme)
    {
        TimeSchemeCoeffs c;

        if (scheme == "backwardEuler")
        {
```

```
            c.theta = 1.0;
            c.bdfOrder = 0;
            return c;
        }

        if (scheme == "crankNicolson")
        {
            c.theta = 0.5;
            c.bdfOrder = 0;
            return c;
        }

        // BDF is fully implicit in K, so theta = 1, and the source is wanted at
        // t^{n+1} only - one source evaluation per step FEWER than
        // Crank-Nicolson needs.
        c.theta = 1.0;

        if (scheme == "BDF1")
        {
            c.bdfOrder = 1;
            c.a[0] =  1.0;
            c.a[1] = -1.0;
        }
        else if (scheme == "BDF2")
        {
            c.bdfOrder = 2;
            c.a[0] =  3.0/2.0;
            c.a[1] = -2.0;
            c.a[2] =  1.0/2.0;
        }
        else if (scheme == "BDF3")
        {
            c.bdfOrder = 3;
            c.a[0] =  11.0/6.0;
            c.a[1] = -3.0;
            c.a[2] =  3.0/2.0;
            c.a[3] = -1.0/3.0;
        }
        else if (scheme == "BDF4")
        {
            c.bdfOrder = 4;
            c.a[0] =  25.0/12.0;
            c.a[1] = -4.0;
            c.a[2] =  3.0;
            c.a[3] = -4.0/3.0;
            c.a[4] =  1.0/4.0;
        }
        else
        {
            FatalErrorInFunction
                << "Unknown implicitScheme: " << scheme << nl
                << "Valid options are backwardEuler, crankNicolson, BDF1,"
                << " BDF2, BDF3 and BDF4." << nl
                << "ESDIRK is not a scheme here: it exists only to supply the"
```

Listing 7: Coeficientes de los esquemas (compartido). `[monodomainTimeSchemes.H]` *(recortado; sigue en el fuente.)*

```
    EsdirkTableau esdirk3Tableau()
    {
        EsdirkTableau tb;
        tb.nStages = 4;
        tb.order = 3;
        tb.gamma = 0.435866521508458999416;

        forAll(tb.c, i)
        {
            tb.c[i] = 0.0;
            forAll(tb.a[i], j)
            {
                tb.a[i][j] = 0.0;
            }
        }

        const scalar g = tb.gamma;
```

```

        tb.c[0] = 0.0;
        tb.c[1] = 2.0*g;
        tb.c[2] = 0.6;
        tb.c[3] = 1.0;

        tb.a[1][0] = g;
        tb.a[1][1] = g;

        tb.a[2][0] =  0.257648246066427245800;
        tb.a[2][1] = -0.093514767574886245216;
        tb.a[2][2] =  g;

        tb.a[3][0] =  0.187641024346723825161;
        tb.a[3][1] = -0.595297473576954948048;
        tb.a[3][2] =  0.971789927721772123471;
        tb.a[3][3] =  g;

        return tb;
    }
```

Listing 8: La tabla del arranque ESDIRK3 (compartido). [monodomainTimeSchemes.H]

```
    List<scalar> applyStimulus
    (
        const scalar t0,
        const scalar dt,
        volScalarField& externalStimulusCurrent,
        const List<labelList>& stimulusCellIDsList,
        const List<scalar>& stimulusStartTimes,
        const scalar stimulusIntensity,
        const scalar stimulusDuration
    )
    {
        scalarField& extI = externalStimulusCurrent.primitiveFieldRef();
        extI = 0.0;

        List<scalar> fraction(stimulusCellIDsList.size(), 0.0);

        forAll(stimulusCellIDsList, bI)
        {
            const scalar tStart = stimulusStartTimes[bI];
            const scalar tEnd = tStart + stimulusDuration;
            const scalar overlap = min(t0 + dt, tEnd) - max(t0, tStart);

            if (overlap <= 0.0 || dt <= 0.0)
            {
                continue;
            }

            fraction[bI] = min(overlap/dt, scalar(1));

            const scalar current = fraction[bI]*stimulusIntensity;
            const labelList& stimulusCellIDs = stimulusCellIDsList[bI];
            forAll(stimulusCellIDs, cI)
            {
                const label cellI = stimulusCellIDs[cI];
                if (mag(current) > mag(extI[cellI]))
                {
                    extI[cellI] = current;
                }
            }
        }

        externalStimulusCurrent.correctBoundaryConditions();

        return fraction;
    }
```

Listing 9: El estímulo promediado sobre el paso. Recortado. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

5. Reconstrucciones en los puntos de Gauss

El término fuente es de alto orden porque I_{ion} se evalúa en los puntos de Gauss y se integra con cuadratura: hace falta V_m (y, en el modo CC, los estados) en esos puntos. `reconstructCellFieldsAtIntegrationPoin` lo hace para **K campos a la vez**, recorriendo el estencil MLS una sola vez por celda: cada vecino se ubica una vez (local o remoto) y los coeficientes de gradiente, hessiano y tercera derivada se cargan una vez para todos los campos. La primera versión compartida hacía un campo por llamada y dejó la reconstrucción de estados del MMS un 30 % más lenta; para el TNNP, con 19 estados, la ganancia fue mayor. El resultado es bit a bit igual a reconstruir de a uno.

`reconstructVmLevelAtIonIntegrationPoints` lleva a los puntos de Gauss un nivel guardado de V_m (de la historia BDF o de una etapa ESDIRK). Es lo que usa `stateODEGaussPointHistory`: con ella, los caminos de puntos de Gauss (GP y el frente del híbrido) usan el mismo polinomio de la historia que el camino por celda, en vez de la recta entre V^n y el candidato. Sin ella BDF3 y BDF4 quedaban en $\sim$2.4 en esos caminos; con ella dan su orden (en el MMS, con solución exacta, BDF4 3.89/3.91), por un 6–9 % de tiempo (**E36**).

```
    void reconstructCellFieldsAtIntegrationPoints
    (
        const fvMesh& mesh,
        const UList<const scalarField*>& fields,
        const highOrderInterp& LREInterp,
        const CompactListList<point>& cellQuadPoints,
        const Writer& write,
        const boolList* writeCellMask,
        const bool limitToNeighbourRange,
        const bool useOpenMP,
        const label openMPThreshold,
        scalar* haloWallTime = nullptr,
        scalar* loopWallTime = nullptr
    )
    {
        const vectorField& C = mesh.C();
        const bool twoD = mesh.nGeometricD() == 2;
        const label nCells = mesh.nCells();
        const label order = LREInterp.order();
        const label nFields = fields.size();

        // Where each cell's quadrature points start in the flat array, taken
        // from the container so that no running counter ties the iterations.
        const labelList& ipOffsets = cellQuadPoints.offsets();

        const auto tHaloStart = std::chrono::steady_clock::now();

        // Halo exchange: MPI, once per field, outside the threaded region.
        const auto& stencilData = LREInterp.mls().stencilData();
        const globalIndex& globalCells = stencilData.globalCells();
        const Map<FixedList<label, 2>>& remoteLoc =
            stencilData.remoteCellLocation();
        const CompactListList<label>& stencils = stencilData.cellsStencil();
        List<List<Field<scalar>>> remote(nFields);
        for (label k = 0; k < nFields; ++k)
        {
            remote[k] = stencilData.remoteFieldPerProc(*fields[k]);
        }

        // The limiter's admissible range must include neighbours on the far
        // side of a processor face: cellCells() stops at the cut, and a cell
        // next to one would otherwise be clamped harder than in serial.
        List<scalarField> coupledLo;
        List<scalarField> coupledHi;
        if (limitToNeighbourRange)
        {
            coupledLo.setSize(nFields, scalarField(nCells, GREAT));
            coupledHi.setSize(nFields, scalarField(nCells, -GREAT));

            if (Pstream::parRun())
            {
                for (label k = 0; k < nFields; ++k)
                {
```

```
                    List<scalar> nbrValues;
                    syncTools::swapBoundaryCellList(mesh, *fields[k], nbrValues);

                    forAll(mesh.boundary(), patchI)
                    {
                        const fvPatch& p = mesh.boundary()[patchI];
                        if (!p.coupled())
```

Listing 10: El núcleo de K campos (compartido). Recortado. `[monodomainReconstruction.H]` *(recortado; sigue en el fuente.)*

```
    void reconstructVmLevelAtIionIntegrationPoints
    (
        const scalarField& cellValues,
        volScalarField& scratch,
        const Switch useHighOrderVm,
        const highOrderInterp& LREInterp_Vm,
        const highOrderInterp& LREInterp_Iion,
        scalarField& VmLevelIntegrationPoints,
        const Switch useOpenMP,
        const label openMPThreshold,
        const Switch profileTimings,
        FineGrainedTimings& fineTimings
    )
    {
        scratch.primitiveFieldRef() = cellValues;
        scratch.correctBoundaryConditions();
        reconstructVmAtIionIntegrationPoints
        (
            scratch,
            useHighOrderVm,
            LREInterp_Vm,
            LREInterp_Iion,
            VmLevelIntegrationPoints,
            useOpenMP,
            openMPThreshold,
            profileTimings,
            fineTimings
        );
    }
```

Listing 11: Un nivel de la historia en los puntos de Gauss (compartido). `[monodomainReconstruction.H]`

```
    void reconstructStatesAtIionIntegrationPoints
    (
        const fvMesh& mesh,
        const Field<Field<scalar>>& statesCells,
        const label nStates,
        const highOrderInterp& LREInterp_states,
        const highOrderInterp& LREInterp_Iion,
        Field<Field<scalar>>& statesIP,
        const boolList* writeCellMask,
        const bool limitToNeighbourRange,
        const Switch useOpenMP,
        const label openMPThreshold
    )
    {
        const CompactListList<point>& qp = LREInterp_Iion.cellQuadPoints();

        // One contiguous column per state: the core reads each field by cell.
        List<scalarField> columns(nStates, scalarField(statesCells.size()));
        List<const scalarField*> fields(nStates);
        for (label stateI = 0; stateI < nStates; ++stateI)
        {
            scalarField& col = columns[stateI];
            forAll(col, cellI)
            {
                col[cellI] = statesCells[cellI][stateI];
            }
            fields[stateI] = &col;
        }

        auto write = [&statesIP](const label ip, const label k, const scalar v)
```

```
        {
            statesIP[ip][k] = v;
        };

        reconstructCellFieldsAtIntegrationPoints
        (
            mesh,
            fields,
            LREInterp_states,
            qp,
            write,
            writeCellMask,
            limitToNeighbourRange,
            useOpenMP,
            openMPThreshold
        );
    }
```

Listing 12: Los estados del TNNP, todos en una pasada. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

6. El residuo de estados: una colectiva que colgaba

`maxStateRelativeL2Difference` da el cambio relativo máximo de los estados entre dos iteraciones no lineales. Es **colectiva**: reduce dos veces por estado. La versión anterior devolvía temprano cuando el arreglo de estados de un rank venía vacío — lo que le pasa a un rank sin celdas — y ese rank salía sin participar de las reducciones: la corrida no terminaba. Ahora el rank vacío llega igual a los dos `reduce`, y el conteo de vueltas se reduce con `minOp`. Lo mide **E32**.

```
    scalar maxStateRelativeL2Difference
    (
        const Field<Field<scalar>>& current,
        const Field<Field<scalar>>& previous,
        scalarField& stateResiduals
    )
    {
        stateResiduals = 0.0;

        // Modified for cardiacFoam: this function is COLLECTIVE - it reduces
        // twice per state below - so every rank has to reach every reduction,
        // and reach it the same number of times.
        //
        // What was here before returned 0.0 early when a rank's state array came
        // up empty, which is exactly what happens to a rank whose partition has
        // no cells. That rank skipped both reductions while the others blocked
        // in them: not a wrong number, a HANG. Reproduced on a 2-cell mesh with
        // `simple (4 1 1)` and with `scotch` at np=8 - both killed at the
        // timeout, while the same case at np=1 finishes in one second (test
        // E19).
        //
        // So the empty case is now handled by letting the accumulation loops
        // below contribute nothing: `forAll(current, ...)` over an empty field
        // iterates zero times, num and den stay 0, and the rank still takes part
        // in every reduce.
        //
        // The trip count needs the same treatment. `current[0].size()` is
        // rank-local and cannot even be read on an empty rank, so it is reduced:
        // otherwise two ranks could run this loop a different number of times
        // and desynchronise the reductions for the same reason.
        label nStates = stateResiduals.size();
        if (!current.empty())
        {
            nStates = min(current[0].size(), stateResiduals.size());
        }
        reduce(nStates, minOp<label>());

        scalar maxResidual = 0.0;

        for (label stateI = 0; stateI < nStates; ++stateI)
        {
```

```
            scalar num = 0.0;
            scalar den = 0.0;

            forAll(current, pointI)
            {
                if (stateI >= current[pointI].size() || stateI >= previous[pointI].size())
                {
                    continue;
                }

                const scalar c = current[pointI][stateI];
                const scalar diff = c - previous[pointI][stateI];
                num += diff*diff;
                den += c*c;
            }

            reduce(num, sumOp<scalar>());
            reduce(den, sumOp<scalar>());
```

Listing 13: La colectiva, con las dos correcciones. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

7. El criterio de parada no lineal, común a todos los métodos

Picard, diagonalIon, JFNK y las etapas ESDIRK paran con el mismo test, `nonlinearIterationExit`: residuo acoplado de la ecuación del paso más los incrementos de V_m, I_{ion} y, si se pide, de los estados. El MMS paraba Picard sólo por incrementos; un test para todos hace que una comparación de métodos (**E08**) compare métodos y no reglas de parada.

Tiene una salida por **piso**: si los incrementos ya pasan y el residuo acoplado dejó de bajar, la iteración se corta y se informa. Se agregó cuando, con el criterio común, Picard del MMS repetía 2000 veces la misma solución lineal con el residuo clavado en $3.66 \cdot 10^{-12}$ contra 10^{-12}. La causa era otra — el KSP ensamblado paraba sobre el residuo *precondicionado*, y ahora precondiciona por derecha — pero el piso queda como red de seguridad.

```
    NonlinearExit nonlinearIterationExit
    (
        const label corr,
        const label minIterations,
        const scalar coupledResidual,
        const scalar coupledResidualPrevIter,
        const scalar coupledTolerance,
        const scalar VmIncrement,
        const scalar VmTolerance,
        const scalar IionIncrement,
        const scalar IionTolerance,
        const bool requireStates,
        const scalar maxStateIncrement,
        const scalar statesTolerance
    )
    {
        const bool incrementsPass =
            corr + 1 >= minIterations
         && VmIncrement <= VmTolerance
         && IionIncrement <= IionTolerance
         && (!requireStates || maxStateIncrement <= statesTolerance);

        if (!incrementsPass)
        {
            return NonlinearExit::Continue;
        }
        if (coupledResidual <= coupledTolerance)
        {
            return NonlinearExit::Converged;
        }
        if (coupledResidual >= coupledResidualPrevIter)
        {
```

```
            return NonlinearExit::ResidualFloor;
        }
        return NonlinearExit::Continue;
    }
```

Listing 14: El test de parada y la salida por piso (compartido). `[monodomainBase.H]`

```
    bool nonlinearIterationStops
    (
        const NonlinearExit how,
        const scalar coupledResidual,
        const scalar coupledTolerance,
        label& nResidualFloorExits
    )
    {
        if (how == NonlinearExit::ResidualFloor)
        {
            ++nResidualFloorExits;
            Info<< "       Coupled residual at its floor (" << coupledResidual
                << " > nonlinearTolerance " << coupledTolerance
                << ", no longer falling) with the increments converged: "
                << "stopping" << nl;
        }
        return how != NonlinearExit::Continue;
    }
```

Listing 15: El conteo y el aviso de una salida por piso (compartido). `[monodomainBase.H]`

8. Bloques de construcción del ensamblado

Las piezas compartidas del ensamblado están en `monodomainAssembly.H` y `monodomainBase.H`. `addTripletIfNeeded` es el punto por el que pasa toda entrada de la matriz y descarta las menores que `SMALL`.

`exchangeCoupledFaceRows` intercambia con el rank vecino las filas que el término de estabilización necesita del otro lado del corte. Es **colectiva** y está deliberadamente *fuera* del lazo de chunks: adentro se trabaría en cuanto dos ranks tuvieran distinta cantidad de chunks.

```
    void addTripletIfNeeded
    (
        std::vector<Triplet>& triplets,
        const label row,
        const label col,
        const scalar value
    )
    {
        if (mag(value) > SMALL)
        {
            triplets.emplace_back(row, col, value);
        }
    }
```

Listing 16: Por acá pasa toda entrada (compartido). `[monodomainBase.H]`

```
    void exchangeCoupledFaceRows
    (
        const fvMesh& mesh,
        const std::function
        <
            void(const label patchI, const label faceI, labelList&, scalarField&)
        >& buildRow,
        List<labelList>& nbrCols,
        List<scalarField>& nbrCoeffs
    )
    {
        nbrCols.setSize(mesh.nBoundaryFaces());
        nbrCoeffs.setSize(mesh.nBoundaryFaces());

        if (!Pstream::parRun())
```

```
        {
            return;
        }

        PstreamBuffers pBufs(Pstream::commsTypes::nonBlocking);

        forAll(mesh.boundary(), patchI)
        {
            const fvPatch& patch = mesh.boundary()[patchI];
            const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);

            if (!ppPtr)
            {
                continue;
            }

            List<labelList> sendCols(patch.size());
            List<scalarField> sendCoeffs(patch.size());

            forAll(patch, faceI)
            {
                buildRow(patchI, faceI, sendCols[faceI], sendCoeffs[faceI]);
            }

            UOPstream toNbr(ppPtr->neighbProcNo(), pBufs);
            toNbr << sendCols << sendCoeffs;
        }

        pBufs.finishedSends();

        forAll(mesh.boundary(), patchI)
        {
            const fvPatch& patch = mesh.boundary()[patchI];
            const processorFvPatch* ppPtr = isA<processorFvPatch>(patch);

            if (!ppPtr)
            {
                continue;
            }

            List<labelList> recvCols;
            List<scalarField> recvCoeffs;

            UIPstream fromNbr(ppPtr->neighbProcNo(), pBufs);
            fromNbr >> recvCols >> recvCoeffs;
```

Listing 17: El intercambio a través del corte (compartido). Recortado. `[monodomainAssembly.H]` *(recortado; sigue en el fuente.)*

9. Las dos matrices de rigidez, y el defecto de la conductividad

Hay dos ensamblados y la tríada elige cuál corre: el ortogonal de dos puntos y el de alto orden sobre el estencil LRE, los dos compartidos con el MMS.

El camino de alto orden usaba `conductivity[own]` en las dos filas de una cara interna, en vez del promedio $\frac{1}{2}(D_{own} + D_{nei})$. Con D uniforme la diferencia se cancela; con D no uniforme hacía que K cambiara un **21 %** al descomponer la malla. El arreglo fueron tres cambios: el promedio en la cara interna, el promedio también en la de procesador, y sacar el intercambio de conductividad de la condición de estabilización. **M15** lo mide: de 2.108e-01 a 4.736e-15.

```
    void assembleHighOrderStiffnessMatrix
    (
        const fvMesh& mesh,
        const volTensorField& conductivity,
        const highOrderInterp& LREInterp,
        const scalar stabilisationAlpha,
        const label tripletsPerFaceReserve,
        const label faceChunk,
        SpMat& K
```

```
    )
    {
        const bool twoD = mesh.nGeometricD() == 2;
        const vectorField& C = mesh.C();
        const surfaceVectorField& Cf = mesh.Cf();
        const CompactListList<point>& faceQP = LREInterp.faceQuadPoints();
        const CompactListList<scalar>& faceQW = LREInterp.faceQuadWeightPhysical();
        const CompactListList<label>& faceStencils =
            LREInterp.globalFaceStencils();
        const List<CompactListList<vector>>& faceGradCoeffs =
            LREInterp.QRGradFaceGPCoeffs();

        const scalarField& V = mesh.V();
        const labelUList& owner = mesh.owner();
        const labelUList& neighbour = mesh.neighbour();

        const label nCells = mesh.nCells();
        const label nInternalFaces = neighbour.size();

        std::vector<Triplet> triplets;
        const label reserveFaces = min(max(mesh.nFaces(), label(1)), faceChunk);
        triplets.reserve(reserveFaces*tripletsPerFaceReserve);

        SpMat Klocal(nCells, gNCols());
        bool kInitialised = false;

        auto flushChunk =
        [&]()
        {
            if (triplets.empty())
            {
                return;
            }

            Klocal.setZero();
            Klocal.setFromTriplets(triplets.begin(), triplets.end());
            Klocal.makeCompressed();

            if (!kInitialised)
            {
                K.swap(Klocal);
                Klocal.resize(nCells, gNCols());
                kInitialised = true;
            }
            else
            {
                K += Klocal;
            }

            triplets.clear();
        };
```

Listing 18: El ensamblado de alto orden (compartido). Recortado. `[monodomainAssembly.H]` *(recortado; sigue en el fuente.)*

10. La solución lineal: PETSc con precondicionado por derecha

`PetscKspMatrixSolver` guarda la matriz, el KSP y la factorización entre pasos, y decide en cada solve si reconstruir, refrescar valores o reusar. En paralelo, `ilu`/`lu` se vuelven block-Jacobi con esa factorización por bloque. Desde 2026-10, GMRES y BiCGStab precondicionan **por derecha** (y cg monitorea la norma sin precondicionar): así la tolerancia lineal mide el mismo residuo verdadero que el criterio no lineal.

```
            // Stop on the TRUE residual, the one the nonlinear stopping test
            // measures. GMRES and BiCGStab default to left preconditioning,
            // whose tolerance applies to the preconditioned residual: with ilu
            // at a large dt the two differed by ~4x, the linear solve
            // "converged" to 1e-12 and left a true residual of 3.66e-12, and
            // Picard repeated that solve up to the iteration cap. Right
            // preconditioning, as the JFNK shell solver already uses; cg has no
```

```
            // right side, so it monitors the unpreconditioned norm instead.
            // Before KSPSetFromOptions, so PETSc options can still override.
            if (isGmresType(kspType) || kspName == "bcgs")
            {
                checkPetscError(KSPSetPCSide(ksp_, PC_RIGHT), "KSPSetPCSide");
            }
            else if (kspName == "cg")
            {
                checkPetscError
                (
                    KSPSetNormType(ksp_, KSP_NORM_UNPRECONDITIONED),
                    "KSPSetNormType"
                );
            }
```

Listing 19: El lado del precondicionador en PetscKspMatrixSolver::reset (compartido). [monodomainPetsc.H]

11. Modo híbrido: GP en el frente, CC en el resto

`flagFrontCells` marca como frente las celdas cuyo V_m varía más de `hybridFrontVmRange` entre sus puntos de Gauss (o que están estimuladas), y `dilateFrontCells` agranda la banda unas capas. La clasificación se calcula **una vez por paso** y se congela para todo el solve no lineal: recalcularla por iteración convierte el punto fijo en un ciclo límite.

En las celdas del frente la EDO se integra en cada punto de Gauss; en el resto, una vez en el centro, como en CC puro. **Desde 2026-10-16 las celdas suaves guardan su propio estado en el centro** (`hybridCellStates`) y sólo el frente vive en los puntos de Gauss. Antes, cada paso rehacían su estado como el promedio de su reconstrucción en Gauss — que no es la identidad: agrega $(h^2/24)\,\Delta s$ — y eso era una difusión de los estados aplicada una vez por paso, más fuerte cuanto más chico Δt: un piso de $2.2 \cdot 10^{-3}$ en la convergencia temporal que ningún esquema podía bajar. Con el arreglo el híbrido da lo mismo que GP con historia (CN orden 2.00, BDF4 5.8e-6 en el par fino) a la mitad del costo, sin limitador.

Al final de cada corrida con híbrido el log informa cuántas veces cambiaron de modo las celdas.

```
    void flagFrontCells
    (
        const fvMesh& mesh,
        const scalarField& VmIntegrationPoints,
        const highOrderInterp& LREInterp_Iion,
        const volScalarField& externalStimulusCurrent,
        const scalar vmRangeThreshold,
        boolList& frontCell
    )
    {
        const CompactListList<point>& cellQP = LREInterp_Iion.cellQuadPoints();
        label ip = 0;
        forAll(mesh.cells(), cellI)
        {
            scalar vMin = GREAT, vMax = -GREAT;
            forAll(cellQP[cellI], qI)
            {
                const scalar v = VmIntegrationPoints[ip];
                vMin = min(vMin, v);
                vMax = max(vMax, v);
                ++ip;
            }
            frontCell[cellI] =
                (vMax - vMin > vmRangeThreshold)
             || (mag(externalStimulusCurrent[cellI]) > SMALL);
        }
    }
```

Listing 20: La clasificación del frente. Recortada. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

```
    void dilateFrontCells
    (
        const fvMesh& mesh,
        const label nLayers,
        boolList& frontCell
    )
    {
        if (nLayers <= 0)
        {
            return;
        }

        const labelListList& cellCells = mesh.cellCells();

        // Added for cardiacFoam: marker field used to carry the front flag
        // across processor boundaries. mesh.cellCells() stops at a processor
        // face, so without this the front band simply failed to grow across a
        // partition. Unlike the stabilisation gap this is an O(1) error, not a
        // shrinking surface effect: a cell classified front integrates a
        // different set of ODEs from one classified smooth.
        //
        // Constructing the field with zeroGradient does NOT put zeroGradient on
        // the processor patches: fvPatchField::New overrides the requested type
        // with the patch's own constraint type, so processor patches receive a
        // real processorFvPatchScalarField and correctBoundaryConditions()
        // performs the halo exchange. One exchange per dilation layer.
        autoPtr<volScalarField> markerPtr;
        if (Pstream::parRun())
        {
            markerPtr.reset
            (
                new volScalarField
                (
                    IOobject
                    (
                        "frontCellMarker",
                        mesh.time().timeName(),
                        mesh,
                        IOobject::NO_READ,
                        IOobject::NO_WRITE
                    ),
                    mesh,
                    dimensionedScalar("zero", dimless, 0.0),
                    zeroGradientFvPatchScalarField::typeName
                )
            );
        }

        for (label layer = 0; layer < nLayers; ++layer)
        {
            boolList grown(frontCell);
            forAll(frontCell, cellI)
            {
                if (!frontCell[cellI])
                {
                    continue;
                }
                const labelList& nbr = cellCells[cellI];
                forAll(nbr, j)
                {
```

Listing 21: La dilatación de la banda. Recortada. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

```
    void averageStatesIntegrationPointsToCells
    (
        const Field<Field<scalar>>& statesIP,
        const label nStates,
        const highOrderInterp& LREInterp_Iion,
        Field<Field<scalar>>& statesCells,
        const boolList* cellMask = nullptr
    )
    {
```

```
        const CompactListList<scalar>& cellIionQuadW =
            LREInterp_Iion.cellQuadWeightPhysical();

        label integrationPointI = 0;
        forAll(cellIionQuadW, cellI)
        {
            const bool updateThisCell =
                (cellMask == nullptr) || (*cellMask)[cellI];

            if (!updateThisCell)
            {
                integrationPointI += cellIionQuadW[cellI].size();
                continue;
            }

            Field<scalar>& sc = statesCells[cellI];
            sc = 0.0;
            scalar wSum = 0.0;

            forAll(cellIionQuadW[cellI], qI)
            {
                const scalar w = cellIionQuadW[cellI][qI];
                const Field<scalar>& sq = statesIP[integrationPointI];
                for (label k = 0; k < nStates; ++k)
                {
                    sc[k] += w*sq[k];
                }
                wSum += w;
                ++integrationPointI;
            }

            const scalar invW = 1.0/max(wSum, SMALL);
            for (label k = 0; k < nStates; ++k)
            {
                sc[k] *= invW;
            }
        }
    }
```

Listing 22: El promedio de Gauss a celda, que ahora se usa una sola vez por celda. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

12. Tiempos de activación y muestreo del benchmark

Lo que el benchmark reporta son **tiempos de activación**. `updateActivationTimes` los interpola *dentro* del paso — $t_n + w\,\Delta t$ con w el cruce lineal del umbral —, así que no quedan cuantizados a múltiplos de Δt; esa interpolación tiene su propio error $O(\Delta t^2)$, y por eso E02 mide órdenes sobre el campo V_m y no sobre los tiempos.

`writeActivationSamples` escribe el valor en P8 y un perfil sobre la diagonal, en puntos físicos fijos muestreados por distancia inversa. `nearestCellToPoint` devuelve el índice en **exactamente un** rank y -1 en el resto; el banner que imprimía el punto de parada no estaba guardado contra ese -1 y daba un SIGFPE aparentemente arbitrario. **E28** verifica que el frente cruce los cortes de procesador sin déficit.

```
    void updateActivationTimes
    (
        const scalarField& VmOld,
        const volScalarField& Vm,
        volScalarField& activationTime,
        boolList& calculateActivationTime,
        const scalar t0,
        const scalar dt,
        const scalar activationThreshold
    )
    {
        const scalarField& VmI = Vm.primitiveField();
        scalarField& activationTimeI = activationTime.primitiveFieldRef();
```

```
        forAll(activationTimeI, cellI)
        {
            if (!calculateActivationTime[cellI])
            {
                continue;
            }

            if (VmOld[cellI] < activationThreshold && VmI[cellI] >= activationThreshold)
            {
                const scalar denom = VmI[cellI] - VmOld[cellI];
                scalar w = 0.0;
                if (mag(denom) > VSMALL)
                {
                    w = (activationThreshold - VmOld[cellI])/denom;
                }

                w = min(max(w, scalar(0.0)), scalar(1.0));
                activationTimeI[cellI] = t0 + w*dt;
                calculateActivationTime[cellI] = false;
            }
        }

        activationTime.correctBoundaryConditions();
    }
```

Listing 23: La interpolación dentro del paso. Recortada. [highOrderElectroActivationFoamImplicitPETScDistributed.C]

```
    label nearestCellToPoint
    (
        const fvMesh& mesh,
        const point& samplePoint
    )
    {
        const vectorField& C = mesh.C();
        label bestCellI = -1;
        scalar bestD2 = GREAT;

        forAll(C, cellI)
        {
            const scalar d2 = magSqr(C[cellI] - samplePoint);
            if (d2 < bestD2)
            {
                bestD2 = d2;
                bestCellI = cellI;
            }
        }

        // Modified for cardiacFoam: the search above is over LOCAL cells only,
        // so every rank used to believe it owned the sample point. Downstream
        // that meant each rank triggered the stop condition on ITS OWN nearest
        // cell, clipped effectiveEndTime differently, and left the time loop at
        // a different step - some ranks dropping out of the collective solve
        // while the others blocked in it. A hang, preceded by corrupted physics,
        // because effectiveEndTime also sets the final step's dt.
        //
        // Exactly one rank now returns a cell; the others return -1. Ties are
        // broken by lowest rank, so the choice is deterministic and independent
        // of the decomposition. reduce() is preferred over Pstream::broadcast,
        // which broadcasts from masterNo() rather than from an arbitrary root.
        if (Pstream::parRun())
        {
            scalar globalBestD2 = bestD2;
            reduce(globalBestD2, minOp<scalar>());

            label winner =
                (bestCellI >= 0 && bestD2 <= globalBestD2)
              ? Pstream::myProcNo()
              : Pstream::nProcs();
            reduce(winner, minOp<label>());

            if (winner >= Pstream::nProcs())
            {
                FatalErrorInFunction
```

```
                    << "Could not find a cell for point " << samplePoint
                    << " on any rank" << exit(FatalError);
            }

            return (winner == Pstream::myProcNo()) ? bestCellI : -1;
        }

        if (bestCellI < 0)
        {
            FatalErrorInFunction
                << "Could not find a cell for point " << samplePoint
                << exit(FatalError);
        }
```

Listing 24: Exactamente un rank gana; el resto devuelve -1. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]` *(recortado; sigue en el fuente.)*

13. main(): montaje y lazo temporal

`main()` lee malla y diccionarios, construye los operadores LRE, ensambla M y K una sola vez y entra al lazo temporal. Cada paso resuelve un sistema no lineal con Picard, JFNK o diagonalIion; **E08** verifica que los tres lleguen al mismo lugar, y un nombre inválido es `FatalError` (**E31**).

Al agotar las iteraciones sin converger, los tres métodos — en los dos solvers — comparten la semántica: por defecto se descarta el paso y se vuelve a los valores del inicio (y se reinicia la historia BDF), y con `nonlinearAcceptUnconverged` se acepta el iterado. Lo verifica **E34**. Los valores por defecto del harness desde 2026-10-16: sin protección del TNNP, sin limitador de estados, con `stateODEGaussPointHistory`.

```
    scalar computeStableDeltaT
    (
        const volTensorField& conductivity,
        const scalar chiVal,
        const scalar CmVal,
        const scalar CFL,
        const scalar dx,
        const label dim
    )
    {
        const scalar Dxx = max(conductivity.component(tensor::XX)).value();
        const scalar Dyy = max(conductivity.component(tensor::YY)).value();
        const scalar Dzz = max(conductivity.component(tensor::ZZ)).value();
        const scalar Dmax = max(Dxx, max(Dyy, Dzz));
        const scalar DefMax = Dmax/(chiVal*CmVal);

        scalar dtStable = GREAT;
        if (dim > 0 && DefMax > SMALL)
        {
            dtStable = CFL*sqr(dx)/(scalar(dim)*DefMax);
        }

        reduce(dtStable, minOp<scalar>());
        return dtStable;
    }
```

Listing 25: El paso estable de referencia. `[highOrderElectroActivationFoamImplicitPETScDistributed.C]`

14. Índice de funciones

Todas las funciones, archivo por archivo y en orden de aparición: primero las del solver, después las de los headers compartidos de `src/monodomainCommon`. Las secciones anteriores recorren las principales; ésta es el mapa completo.

highOrderElectroActivationFoamImplicitPETScDistributed.C

línea	función	líneas
374	embeddedComputeStimulus	43
418	computeIstim	14
787	tissueFlag	14
802	stateIndex	8
811	algebraicIndex	8
820	loadStimulusProtocol	10
836	recordCorrection	12
849	protectState	57
909	drivenVmMilliVolt	11
927	computeRates	51
979	computeVariables	19
999	eulerStep	19
1019	rk4Step	31
1051	rkf45Trial	41
1098	advancePoint	114
1218	emptyScratchIDs	5
1226	setOpenMP	5
1385	initialiseStates	32
1418	updateStatesOld	4
1423	resetStatesToStatesOld	4
1428	calculateCurrent	39
1468	solveODE	154
1623	exportStates	31
1655	exportStatesIntegrationPoints	38
1700	characteristicDx	24
1729	computeStableDeltaT	25
1776	applyStimulus	45
1827	maxStateRelativeL2Difference	66
1906	postProcessingDir	11
1938	reconstructStatesAtIionIntegrationPoints	47
1990	averageIionIntegrationPointsToCells	34
2029	averageStatesIntegrationPointsToCells	47
2084	flagFrontCells	27
2121	dilateFrontCells	116
2247	updateActivationTimes	38
2288	nearestCellToPoint	62
2356	sampleIDW	146
2506	writeActivationSamples	63
2571	main	2984

monodomainAssembly.H

línea	función	líneas
30	exchangeCoupledFaceRows	78
113	orthogonalDiffusionCoeff	14
130	normalDiffusivity	4
140	stabilisationFaceCoeff	17

línea	función	líneas
193	nonOrthFluxVector	30
233	computeHighOrderLaplacian	80
317	addDiagonalToMatrix	18
338	assembleDiagonalMassMatrix	31
373	addCellGradientDotCoeffs	58
434	buildCellGradientDotRow	26
477	cellTaylorEntryCoeff	32
513	buildCellTaylorExtrapolationRow	29
550	addCellTaylorExtrapolationCoeffs	57
619	addRhieChowInternalJumpCoeffs	34
660	addRhieChowCoupledJumpCoeffs	38
706	assembleConsistentMassMatrixHO	237
968	assembleStandardOrthogonalStiffnessMatrix	412
1400	assembleHighOrderStiffnessMatrix	445

monodomainBase.H

línea	función	líneas
33	globalRowStart	14
55	gCol	4
61	gNCols	4
70	initialiseGlobalCells	6
87	requireOneOf	31
126	gSquaredNorm	6
135	gNorm	4
143	gDot	6
153	fieldToEigVec	12
169	eigVecToField	9
181	relativeL2Norm	5
191	addTripletIfNeeded	13
207	addRemoteRowCoeffs	19
239	clampPhysical	13
258	relativeL2Difference	29
301	reportOperatorFingerprint	35
357	nonlinearIterationExit	36
396	nonlinearIterationStops	18
468	currentPeakRSSMB	14
486	logMemoryCheckpoint	5
494	normalizedMassMatrixType	5
505	memoryOptimizationIsEffective	22

monodomainKrylov.H

línea	función	líneas
29	diagonalBlock	23
66	solveGMRES	142
315	beginStep	4

línea	función	líneas
329	update	92
423	apply	25
483	solveJfnkLinearSystem	44
569	solveSparseSystemEigen	94
672	solveSparseSystem	101

monodomainPetsc.H

línea	función	líneas
20	checkPetscError	10
32	copyPetscVecToEigVec	16
60	buildDistributedMat	85
177	clear	15
193	reset	19
306	setPetscOptionsPrefix	11
327	usesPetscBackend	5
344	resolvePetscKspPc	18
373	petscKspTypeForLinearSolver	12
395	petscParallelPcTypeName	13
411	isGmresType	4
428	copyEigVecToPetscVec	12
706	updateValues	69
776	solve	57
846	petscShellMatMult	17
866	petscShellPCApply	17
954	initialise	172

monodomainReconstruction.H

línea	función	líneas
17	quadraticForm	10
35	reconstructFromTaylor	24
99	reconstructCellFieldsAtIntegrationPoints	249
355	reconstructVmAtIionIntegrationPoints	73
436	reconstructVmLevelAtIionIntegrationPoints	29

monodomainTimeSchemes.H

línea	función	líneas
77	esdirk3Tableau	37
117	timeSchemeCoeffs	71
217	interpolateVmDriver	31